

First-In, First-Out Page Replacement Algorithm

1 Core Idea

Definition

The **First-In, First-Out (FIFO)** page replacement algorithm evicts the page that has been in memory the longest.

FIFO treats memory like a queue. New pages enter at the tail of the queue. When a page must be evicted, the page at the head of the queue is removed.

FIFO Rule

evict the oldest page in memory

2 Queue Structure

The operating system keeps a list of pages currently in memory.

Position	Meaning
Head	Least recent arrival; this page entered memory earliest.
Tail	Most recent arrival; this page entered memory latest.

Queue Movement

On a page fault, remove from the head and insert the incoming page at the tail.

3 Algorithm Steps

1. A page fault occurs.
2. If a free frame exists, load the needed page into it.
3. If memory is full, choose the page at the head of the FIFO queue.
4. Evict that oldest page.
5. Load the new page.
6. Add the new page to the tail of the queue.

4 Worked Example

Problem

Memory has 3 frames. The FIFO queue is:

Head $[A, B, C]$ Tail

A page fault occurs for page D . Which page is evicted?

Solution

The page at the head is A , so A is the oldest page in memory.

$$[A, B, C] \rightarrow [B, C, D]$$

Answer

FIFO evicts A and inserts D at the tail.

5 Supermarket Analogy

FIFO is like a store with limited shelf space.

Store idea	Memory idea
Shelves can hold only k products.	RAM can hold only k pages.
New product arrives.	New page must be loaded.
Oldest stocked product is removed.	Oldest resident page is evicted.

The analogy shows why FIFO is simple: the system only needs to remember arrival order.

6 Main Weakness

Problem

The oldest page is not necessarily the least useful page.

A page may have entered memory long ago and still be heavily used. FIFO does not check whether the page was referenced recently.

FIFO assumption	Why it can fail
Old pages are good victims.	An old page may still contain important code, data, or stack information.
Arrival time is enough.	FIFO ignores actual page usage after arrival.

Key Point

FIFO has low overhead, but pure FIFO is rarely used because it can evict useful pages.

7 Strengths and Weaknesses

Point	Explanation
Strength	Very easy to implement.
Strength	Low overhead: only a queue of resident pages is needed.
Weakness	Ignores whether a page is actively used.
Weakness	May remove a heavily used page just because it is old.

8 Final Summary

One-Box Summary

FIFO evicts the page that entered memory earliest.
It stores pages in arrival order: head = oldest, tail = newest.
It is simple and cheap, but it ignores actual page usage.